

Keepers of Quality — A Code Health and Technical Debt Dashboard for Video Game Development

Supervisor: Stefano Lambiase (stla@cs.aau.dk)

Project Description

Video game development is, at its core, a software engineering activity — but a markedly peculiar one. Studios ship codebases that mix multiple languages (C++, C#, scripting layers), rely heavily on engine-specific idioms (Unity, Unreal, Godot), depend on large asset pipelines alongside the source code itself, and evolve under a creative process in which “fun” is the implicit, ever-shifting requirement. These conditions create software engineering problems that differ substantially from those found in typical enterprise or web-service settings, and they often escape the assumptions baked into general-purpose development tooling.

A growing body of empirical work has documented that technical debt (TD) in video game codebases is both real and systemically under-managed. Industry surveys report crunch, brittle architectures, and broken builds, together with a distinct lack of systematic approaches to identifying and addressing code-level quality issues. At the same time, game developers themselves report greater difficulty localizing bugs than their peers in non-game software, and classic software engineering practices such as automated testing and requirements engineering are adopted in highly ad-hoc forms — largely because standard tools and methods do not account for game-specific structures and concerns.

General-purpose code-quality platforms such as SonarQube have matured significantly, yet they remain calibrated to the idioms of enterprise software. They struggle to detect game-specific code smells (tight coupling between scripts and scene objects, inappropriate use of physics and rendering APIs, misuse of coroutines, memory churn in update loops, and so on), they have no awareness of asset pipelines or build/cook health, and they offer no unified view that ties code-level signals to downstream player-facing issues such as stuttering, crashes, or regressions surfaced in post-release gameplay. Recent research on game-specific bad smells — catalogued, validated with practitioners, and implemented in open-source prototypes — demonstrates that this gap is not only real but technically addressable.

The goal of this thesis proposal is to investigate how a game-aware code-quality and technical-debt dashboard can be designed, implemented, and empirically evaluated. The work will start from a requirements-engineering phase grounded in direct engagement with game developers, and will produce a working prototype capable of analyzing a real game codebase (e.g., a Unity project) against a curated catalog of game-specific smells and TD indicators, and of presenting the results in a longitudinal, studio-scale dashboard. The prototype will be validated through user-centered studies with practitioners, assessing its usefulness, usability, and impact on day-to-day development practices. The scope is intentionally broad to allow students to tailor the research focus to their interests — from the static-analysis engine itself, to the visualization and developer-experience layer, to integration with post-release player-facing signals. Students are encouraged to creatively explore how to push the state of the art forward, with the supervisor available for brainstorming and guidance.

Potential collaborations with companies in Denmark can be arranged. The professor already has some contacts with game startups in Denmark, but the plan is to search for collaborations with others.

Interesting References (**important ones are in red**)

Game-Specific Code Smells and Maintainability

- Detecting Video Game-Specific Bad Smells in Unity Projects (MSR 2020, 32 citations). Borrelli, Nardone, Di Lucca, Canfora, and Di Penta introduce UnityLinter, a static analyzer for seven Unity-specific smells spanning performance, maintainability, and incorrect behavior. The catalog was derived from literature and forums, validated with 68 practitioners, and checked on 100 open-source Unity projects. A direct technical foundation for any game-aware analysis engine.
- Video Game Bad Smells: What They Are and How Developers Perceive Them (ACM TOSEM 2023, 20 citations). Nardone et al. expand the earlier work into a 28-smell catalog across five categories (game design/logic, physics, animation, rendering, multiplayer), grounded in the open coding of 572 forum discussions and validated with 76 professionals. The canonical reference for the claim that games possess their own design-smell vocabulary that generic tools miss.
- UnityLint: A Bad Smell Detector for Unity (ICPC 2023). Bosco et al. provide an open-source implementation of the 2023 catalog, detecting 18 smell types in Unity projects. Effectively the reference implementation on top of which a commercial-quality dashboard could be built.
- Analyzing Maintainability Factors in Open-Source Game Engines (Multimedia Tools and Applications 2025). Öztürk applies static analysis and a fuzzy-logic maintainability model to open-source game engines, finding that most (including Godot) score poorly on maintainability — suggesting that generic code-quality methods alone are insufficient for game software.
- Design Patterns and Code Maintainability in Games: A Case Study (SBGames 2023). Estevam et al. run a before/after refactoring study using standard maintainability metrics, showing modest but measurable gains when design patterns are applied to game codebases.

Technical Debt and Software Engineering in Game Development

- Living With Technical Debt — A Perspective From the Video Game Industry (IEEE Software 2021, 7 citations). Borowa et al. survey practitioners on how TD manifests specifically in games, explicitly documenting a “distinct lack of systematic management of technical debt” in the industry. The closest existing evidence that a game-specific TD tool addresses an unmet need.
- Cowboys, Ankle Sprains, and Keepers of Quality: How Is Video Game Development Different From Software Development? (ICSE 2014, 193 citations). Murphy-Hill et al. combine interviews with a 364-respondent survey to show concrete ways game development diverges from classic software engineering (e.g., reluctance to adopt

automated testing because creative churn makes tests obsolete). A foundational “games are different” reference.

- Software Engineering Challenges in Game Development (2009, 136 citations). Kanode and Haddad catalog the software engineering problems unique to games, driven by multi-disciplinary teams (art, music, acting, programming), heavy asset management, and the search for “fun.”
- Bug Localization in Game Software Engineering: Evolving Simulations to Locate Bugs in Software Models of Video Games (MODELS 2022, 9 citations). Casamayor et al. present a game-specific bug localization approach evaluated on a commercial game (Kromaia) and document that developers perceive bug localization as harder in games than in non-game software.
- Requirements Engineering: Foundation for Software Quality (2014, 123 citations). Ferrari et al. interview 27 professionals across seven game organizations and find that requirements engineering practices in games are ad-hoc and dominated by “fun” as an implicit requirement.

Player-Facing and Post-Release Quality Signals

- Using Gameplay Videos for Detecting Issues in Video Games (Empirical Software Engineering 2023, 6 citations). Guglielmi, Scalabrino, Bavota, and Oliveto introduce GELID, which segments and categorizes bug-relevant moments in streamer gameplay videos. Demonstrates how a dashboard could integrate post-release, player-facing signals alongside source-level analyses.
- Automatic Identification of Game Stuttering via Gameplay Videos Analysis (ACM TOSEM 2024). Guglielmi et al. present HASTE, a stuttering / frame-rate-drop detector applied to gameplay videos (71% recall, 89% precision). Shows that performance regressions can be surfaced from public Twitch or YouTube streams without instrumenting the game.

Testing, QA, and Reuse in Games

- A Literature Review of Software Testing Practices and Frameworks in the Video Gaming Industry (Software Testing 2025, 3 citations). Roque et al. provide a systematic review of testing tools and processes in games, flagging the industry's shift from brute-force manual QA toward automation.
- A Comprehensive Overview of Software Quality Assurance for Computer Gaming (PDGC 2024). Shukla et al. survey the state of practice of SQA in games, including the role of AI/ML in automated game testing.
- Evaluating the Benefits of Software Product Lines in Game Software Engineering (SPLC 2022). Trasobares et al., together with a 2023 Journal of Systems and Software follow-up comparing software product lines with clone-and-own practices, show that classical SE reuse practices transfer imperfectly to games — reinforcing the argument that game SE is distinct enough to warrant dedicated tooling.
- Quality of Reusable Game Software: Empowering Developers with Automated Quality Checks (QRS 2019). van der Vegt et al. describe a plug-in-based QA tool for reusable

(serious) game components — an early gesture toward the idea of game-component quality dashboards.

Technical Debt Foundations (general software engineering)

- A Systematic Mapping Study on Technical Debt and Its Management (Journal of Systems and Software 2015, 641 citations). Li, Avgeriou, and Liang provide the canonical mapping study of the TD research field.
- A Tertiary Study on Technical Debt (Information and Software Technology 2018, 191 citations). Alves et al. offer a taxonomy of TD types, activities, and tools — a useful structure for categorizing the dashboard's signals.
- Technical Debt Management: The Road Ahead for Successful Software Delivery (ICSE-FoSE 2023, 16 citations). Avgeriou et al. present a recent vision paper on where TD management is heading, grounding the dashboard design in current research priorities.
- The Influence of Technical Debt on Software Developer Morale (Journal of Systems and Software 2020, 46 citations). Besker et al. link TD signals to developer morale — a natural bridge to the crunch and burnout concerns documented in the games industry.

Consensus search:

<https://consensus.app/search/technical-debt-video-game-development/>